Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ingeniería Electrónica
Departamento de Sistemas e Informática

SISTEMAS EMBEBIDOS AVANZADOS II

FreeRTOS sobre ESP32-S3

Guía de estudio: entorno ESP-IDF, FreeRTOS multinúcleo y ejemplos de tareas, semáforos, mutex, colas y timers

Prof. Ing. Daniel Márquez

Octubre de 2026 — Versión 2.0

Índice

1. Introducción y objetivos 5
1.1. Componentes del entorno 5
1.2. Organización de la guía y de los ejemplos 5
2. Requisitos 6
2.1. Hardware 6
2.2. Software y PC 6
3. Instalación del entorno 6
3.1. Instalar Visual Studio Code 6
3.2. Instalar la extensión ESP-IDF 6
3.3. Instalar ESP-IDF con el EIM 7
3.4. Vincular ESP-IDF con VS Code 7
4. Conexión de la placa 7
5. Creación del proyecto base 7
5.1. Crear el proyecto 7
5.2. El target: cómo sabe el proyecto que el chip es un ESP32-S3 8
5.3. Ajustar el pin del LED (obligatorio para la placa N16R8) 8
5.4. Compilar, grabar y monitorear 8
6. Estructura del proyecto y configuración 9
6.1. Árbol del proyecto 9
6.2. menuconfig, sdkconfig y sdkconfig.h 9
6.3. Configuración de FreeRTOS: FreeRTOSConfig.h 10
7. El LED RGB direccionable (WS2812) y el periférico RMT 11
8. FreeRTOS en ESP-IDF 12
8.1. Secuencia de arranque 12
8.2. app_main() se ejecuta dentro de una tarea 12
8.3. Tareas que crea el sistema 12
8.4. El tick del sistema 13
9. FreeRTOS en dos núcleos (SMP) 13
9.1. Los núcleos 13
9.2. Afinidad: en qué núcleo corre una tarea 13
9.3. Planificación en dos núcleos 14
9.4. Exclusión mutua en dos núcleos 14
9.5. Otras diferencias 14
9.6. Resumen comparativo 15
9.7. En qué núcleo corre cada ejemplo 15
10. Ejemplo 1: blink con app_main() como tarea 16
10.1. Objetivo 16
10.2. Código 16

10.3. Explicación 17
10.4. Salida esperada 18
10.5. Preguntas 18
11. Ejemplo 2: dos tareas 19
11.1. Objetivo 19
11.2. Código 19
11.3. Explicación 20
11.4. Salida esperada 21
11.5. Preguntas 21
12. Ejemplo 3: recurso compartido y semáforo binario 22
12.1. Objetivo y escenario 22
12.2. Conceptos 22
12.3. Código 22
12.4. Explicación 24
12.5. Salida esperada 25
12.6. Preguntas 25
13. Ejemplo 4: inversión de prioridad y mutex 26
13.1. Objetivo 26
13.2. Conceptos 26
13.3. Escenario 26
13.4. Código 26
13.5. Explicación 29
13.6. Salida esperada (semáforo binario) 29
13.7. Preguntas 29
14. Ejemplo 5: cola de mensajes 31
14.1. Objetivo 31
14.2. Conceptos 31
14.3. Código 31
14.4. Explicación 33
14.5. Salida esperada 33
14.6. Para discutir 33
15. Ejemplo 6: cola de mensajes y software timer 34
15.1. Objetivo 34
15.2. Conceptos 34
15.3. Código 34
15.4. Explicación 36
15.5. Resultado esperado 37
15.6. Para discutir 37
16. Resumen de la API utilizada 37

17. Problemas frecuentes 38
18. Actividades propuestas 39
19. Referencias 39

1. Introducción y objetivos

Esta guía acompaña el trabajo práctico de la materia sobre FreeRTOS en el microcontrolador **ESP32-S3** de Espressif. Cubre la instalación del entorno de desarrollo oficial en Windows, la creación y configuración de un proyecto, las particularidades de la versión de FreeRTOS que incluye ESP-IDF —en especial su funcionamiento en **dos núcleos**— y una secuencia de seis ejemplos que introducen, de a uno, los mecanismos fundamentales de un sistema operativo de tiempo real.

Al finalizar, el estudiante debería ser capaz de:

- Instalar ESP-IDF con el **ESP-IDF Installation Manager (EIM)**, crear un proyecto desde VS Code, configurarlo, compilarlo, grabarlo y monitorear su salida.
- Ubicar y modificar la configuración del proyecto y del kernel FreeRTOS (**menuconfig**, `sdkconfig`, `FreeRTOSConfig.h`).
- Explicar por qué en ESP-IDF FreeRTOS ya está corriendo al entrar a `app_main()`, y que esta función se ejecuta dentro de una **tarea**.
- Describir las diferencias entre FreeRTOS estándar y la versión **SMP de dos núcleos** de ESP-IDF, y determinar en qué núcleo puede correr cada tarea.
- Crear tareas, proteger recursos compartidos con **semáforos** y **mutex**, reconocer la **inversión de prioridad**, comunicar tareas con **colas de mensajes** y ejecutar acciones periódicas con **software timers**.

1.1. Componentes del entorno

Componente	Función
ESP-IDF	Framework oficial de Espressif (*IoT Development Framework*). Incluye drivers, bibliotecas, el sistema de compilación basado en CMake y una versión de FreeRTOS adaptada a los chips de Espressif, con soporte para dos núcleos (SMP).
Toolchain	Compilador cruzado GCC para la arquitectura Xtensa del ESP32-S3, depurador y utilidades asociadas.
EIM	*ESP-IDF Installation Manager*. Aplicación que descarga e instala ESP-IDF, el toolchain, Python y demás herramientas, y verifica los prerrequisitos.
VS Code + extensión ESP-IDF	Entorno de edición. La extensión agrega comandos para crear proyectos, configurar, compilar, grabar, monitorear y depurar, pero **no instala ESP-IDF por sí sola**: necesita que el EIM lo haya instalado.
esptool	Programador que se comunica con el bootloader en ROM del chip para grabar la memoria flash.

1.2. Organización de la guía y de los ejemplos

Las secciones 2 a 7 cubren el entorno: instalación, primer proyecto, estructura y configuración. Las secciones 8 y 9 presentan FreeRTOS en ESP-IDF y su funcionamiento en dos núcleos. Las secciones 10 a 15 desarrollan los seis ejemplos, cada uno con su código completo, la explicación de los conceptos nuevos, la salida esperada y preguntas para discutir. Al final se incluyen un resumen de la API, problemas frecuentes, actividades propuestas y referencias.

Todos los ejemplos se ejecutan sobre el mismo proyecto, creado a partir del ejemplo oficial *blink* en la sección 5. Para probar cada uno, basta con reemplazar el contenido del archivo `main/blink_example_main.c` por el código del ejemplo y volver a compilar y grabar. Se conserva el nombre del archivo porque es el que figura en `main/CMakeLists.txt`.

Ejemplo	Tema	Conceptos
1	Blink con `app_main()`	`app_main()` como tarea, `vTaskDelay()`, tick del sistema
2	Dos tareas	`xTaskCreate()`, stack, prioridades, núcleo de ejecución, `app_main()` que retorna
3	Semáforo binario	Recurso compartido, condición de carrera, sección crítica, `xSemaphoreTake()`/`Give()`
4	Mutex e inversión de prioridad	Herencia de prioridad, `xTaskCreatePinnedToCore()`, `vTaskDelayUntil()`
5	Cola de mensajes	Productor/consumidor, copia por valor, bloqueo en cola vacía/llena
6	Cola + software timer	`xTimerCreate()`, callbacks, tarea Tmr Svc

2. Requisitos

2.1. Hardware

- Placa de desarrollo **ESP32-S3** con LED RGB integrado. La placa de la cátedra es del tipo **ESP32-S3 N16R8**: la sigla indica 16 MB de memoria flash (N16) y 8 MB de PSRAM (R8).
- Cable USB **de datos**. Algunos cables solo transmiten energía y la PC no detecta la placa.

2.2. Software y PC

- Windows 10 u 11 de 64 bits, con permisos para instalar programas.
- Conexión a Internet durante la instalación (se descargan varios GB) y al menos 10 GB libres en disco.

> **Importante:** ESP-IDF no tolera rutas con **espacios ni acentos**. Si el usuario de Windows tiene un nombre como `José Pérez`, los proyectos no deben crearse dentro de `C:\Users\José Pérez\`. Se recomienda una carpeta como `C:\esp\` para todos los proyectos de la materia.

3. Instalación del entorno

La instalación consta de tres partes, que deben realizarse en este orden: VS Code, la extensión ESP-IDF y ESP-IDF mediante el EIM. Se recomienda completarla **antes de la primera clase práctica**, ya que la descarga puede demorar entre 10 y 30 minutos.

3.1. Instalar Visual Studio Code

1. Descargar VS Code desde **code.visualstudio.com** e instalarlo con las opciones por defecto.
2. Abrir VS Code para verificar que funciona.

3.2. Instalar la extensión ESP-IDF

1. En VS Code, abrir el panel de **Extensiones** (`Ctrl+Shift+X`).
2. Buscar **ESP-IDF** e instalar la extensión publicada por **Espressif Systems**.
3. En la barra lateral aparece el ícono de Espressif. Al hacer clic se abre el panel **ESP-IDF: Explorer**, con una lista de comandos (**Commands**) y una sección **Advanced**.

> **Atención:** en este punto la extensión está instalada, pero **ESP-IDF todavía no**. Si se ejecuta **Select current ESP-IDF version**, la lista aparece vacía. Es normal: falta el paso siguiente.

3.3. Instalar ESP-IDF con el EIM

El **EIM** es una aplicación independiente de VS Code. Si desde la extensión se ejecuta **Advanced > Open ESP-IDF Installation Manager** sin tenerlo instalado, se obtiene el mensaje **EIM not found**. Por eso se instala primero por separado.

1. Ingresar a la guía oficial de instalación para Windows: **docs.espressif.com/projects/esp-idf/en/stable/get-started/windows-setup.html**.
2. En la sección **Download the EIM**, descargar el instalador para **Windows (x64)** e instalarlo. Como alternativa, la misma página explica cómo hacerlo con el gestor de paquetes **WinGet**.
3. Abrir la aplicación **ESP-IDF Installation Manager** desde el menú Inicio.
4. En **Easy Installation**, hacer clic en **Start Easy Installation**. Esta opción instala la última versión estable de ESP-IDF con la configuración por defecto, y verifica e instala los prerrequisitos que falten.
5. Esperar a que finalice y cerrar el EIM.
6. **Cerrar y volver a abrir VS Code**, para que la extensión detecte la instalación.

3.4. Vincular ESP-IDF con VS Code

1. En **ESP-IDF: Explorer > Commands**, hacer clic en **Select current ESP-IDF version**.
2. En la lista que se despliega arriba debe aparecer la versión instalada. Seleccionarla.
3. Si la lista sigue vacía, repetir el punto 3.3. El comando **Advanced > Doctor Command** genera un reporte de diagnóstico del entorno.

4. Conexión de la placa

La placa tiene dos conectores USB-C. Cualquiera de los dos permite grabar el programa y ver los mensajes en el monitor serie:

Conector	Descripción	Observaciones
USB	USB nativo del ESP32-S3 (controlador USB-Serial/JTAG interno).	No requiere drivers. Permite además **depurar** (breakpoints, paso a paso) sin hardware adicional.
UART / COM	Conversor USB-serie externo (CH343, CP210x u otro, según el fabricante).	En algunas PC requiere instalar el driver del fabricante del conversor.

Para identificar el puerto asignado, abrir el **Administrador de dispositivos** de Windows y desplegar **Puertos (COM y LPT)**. Al conectar la placa aparece un nuevo puerto, por ejemplo `COM5`. La **consola** de ESP-IDF —donde aparecen los mensajes de `ESP_LOGI()` y `printf()`— es ese mismo puerto serie.

5. Creación del proyecto base

En lugar de escribir un proyecto desde cero, se parte del ejemplo oficial **blink**, incluido en ESP-IDF. Este ejemplo hace parpadear el LED de la placa y ya contempla el caso de un LED RGB direccionable como el de nuestra placa. Además, trae declarada la dependencia del componente `led_strip`, que usan varios de los ejemplos.

5.1. Crear el proyecto

1. En **ESP-IDF: Explorer**, ir a **Advanced > New Project Wizard**.
2. En la pestaña **New Project**, escribir `blink` en **Search Template By Name**, o desplegar **ESP-IDF Examples > get-started > blink**.
3. Seleccionar **blink** y completar el formulario según la tabla siguiente.

Campo	Valor	Comentario
Project Name	`blink`	Nombre de la carpeta del proyecto.
Enter Project directory	`C:\esp`	Ruta sin espacios ni acentos. La carpeta debe existir.
Choose ESP-IDF Target (IDF_TARGET)	`esp32s3`	Define el chip para el que se compila.
Choose ESP-IDF Board	ESP32-S3 chip (via builtin USB-JTAG)	Solo afecta la depuración (OpenOCD), no la grabación.
Choose serial port	El COM de la placa	Si se deja en **detect**, se elige luego desde la barra inferior.
Add your ESP-IDF Component directory	Vacío	Opcional. Sirve para agregar bibliotecas propias reutilizables.

4. Hacer clic en **Create Project** y aceptar cuando VS Code ofrezca abrir el proyecto.

5.2. El target: cómo sabe el proyecto que el chip es un ESP32-S3

El proyecto no detecta el chip por sí solo: lo define el **target**. Al elegir `esp32s3` (en el formulario o con **Set Espressif Device Target**), la extensión ejecuta `idf.py set-target esp32s3`. Esto genera el archivo `sdkconfig` con `CONFIG_IDF_TARGET="esp32s3"`, y a partir de allí la compilación usa el toolchain, los drivers y el mapa de memoria del ESP32-S3. El ejemplo incluye además archivos `sdkconfig.defaults.<chip>` con valores por defecto para cada chip; al fijar el target se aplica `sdkconfig.defaults.esp32s3`. Si no se define el target, el proyecto queda configurado para el **ESP32 original**.

Como protección, al grabar, **esptool** lee la identificación del chip conectado y aborta si no coincide con el target, con un mensaje del tipo *This chip is ESP32-S3, not ESP32*.

5.3. Ajustar el pin del LED (obligatorio para la placa N16R8)

El ejemplo trae como pin por defecto para el ESP32-S3 el **GPIO 38**, que corresponde a la placa de referencia de Espressif (ESP32-S3-DevKitC-1 v1.1). En la placa N16R8 de la cátedra, el LED RGB está conectado al **GPIO 48**. Si no se modifica, el programa funciona y envía mensajes por el puerto serie, pero el LED no responde: queda encendido en un color fijo o apagado.

1. En **ESP-IDF: Explorer > Commands**, abrir **SDK Configuration Editor (menuconfig)**.
2. En el buscador escribir `Blink`. Aparece la sección **Example Configuration**.
3. Verificar o configurar los siguientes valores y hacer clic en **Save**:

Opción	Valor	Significado
Blink LED type	LED strip	El LED es direccionable (WS2812), no un LED común.
LED strip backend peripheral	RMT	Periférico que genera la señal del LED.
Blink GPIO number	**48**	Pin del LED en la placa N16R8.
Blink period in ms	1000	Período del ejemplo 1.

5.4. Compilar, grabar y monitorear

Antes de compilar, verificar en **ESP-IDF: Explorer > Commands** (o en la barra inferior de VS Code): **Set Espressif Device Target** en `esp32s3`, **Select Port to Use** con el COM de la placa y **Select Flash Method** en **UART**, que funciona con cualquiera de los dos conectores.

Luego ejecutar **Build, Flash and Monitor**, que compila, graba y abre el monitor serie en un solo paso. La primera compilación demora varios minutos porque se compila todo el framework y se descarga `led_strip`; las siguientes son mucho más rápidas. Para salir del monitor se usa `Ctrl+]`.

Cada línea de log tiene el formato `I (1301) example: Turning the LED ON!`: la letra indica el nivel (**I** = *Info*, **W** = *Warning*, en amarillo; **E** = *Error*, en rojo), el número entre paréntesis es el tiempo en milisegundos desde el arranque y luego aparece la etiqueta (*tag*) del módulo que generó el mensaje.

6. Estructura del proyecto y configuración

6.1. Árbol del proyecto

```
C:\esp\blink\
├── CMakeLists.txt              Proyecto: nombre y sistema de compilación
├── sdkconfig                   Configuración efectiva del proyecto (generada)
├── sdkconfig.defaults          Valores por defecto comunes
├── sdkconfig.defaults.esp32s3  Valores por defecto para el ESP32-S3
├── sdkconfig.defaults.<chip>   ... uno por cada chip soportado
├── main\
│   ├── CMakeLists.txt          Registra los fuentes del componente main
│   ├── Kconfig.projbuild       Define las opciones de 'Example Configuration'
│   ├── idf_component.yml       Dependencias externas (led_strip)
│   └── blink_example_main.c    Código de la aplicación
├── managed_components\         Componentes descargados (generada)
└── build\                      Resultados de compilación (generada)
    └── config\
        └── sdkconfig.h         Las opciones CONFIG_... como macros de C (generado)
```

- Un proyecto ESP-IDF está formado por **componentes**. `main` es el componente de la aplicación; ESP-IDF aporta muchos más (drivers, FreeRTOS, log, etc.), que residen en la carpeta de instalación de ESP-IDF, no en el proyecto.
- `idf_component.yml` declara la dependencia `espressif/led_strip`. El **gestor de componentes** la descarga del registro de Espressif en la primera compilación y la guarda en `managed_components/`.
- `Kconfig.projbuild` define las opciones propias del ejemplo, que aparecen en menuconfig dentro de **Example Configuration**.
- `build/`, `managed_components/` y `sdkconfig` se regeneran: no deben editarse a mano ni, en general, subirse al repositorio (salvo `sdkconfig`, si se quiere conservar la configuración).

6.2. menuconfig, sdkconfig y sdkconfig.h

Cada opción configurable de ESP-IDF tiene un nombre que comienza con `CONFIG_`. **menuconfig** (*SDK Configuration Editor*) es la interfaz para modificarlas; el resultado se guarda en `sdkconfig`. Por ejemplo, la sección del ejemplo blink queda así:

```
# CONFIG_BLINK_LED_GPIO is not set
CONFIG_BLINK_LED_STRIP=y
CONFIG_BLINK_LED_STRIP_BACKEND_RMT=y
# CONFIG_BLINK_LED_STRIP_BACKEND_SPI is not set
CONFIG_BLINK_GPIO=48
CONFIG_BLINK_PERIOD=1000
```

- `=y` indica una opción booleana activada; una desactivada se escribe como comentario: `# ... is not set`. Las opciones numéricas guardan su valor.
- Al compilar, `sdkconfig` se traduce a `build/config/sdkconfig.h`, donde cada opción es una **macro de C**. Por eso el código puede escribir `#define BLINK_GPIO CONFIG_BLINK_GPIO` sin tener el número de pin escrito a mano.

> **Buena práctica:** modificar la configuración siempre desde **menuconfig**, que resuelve las dependencias entre opciones. Editar `sdkconfig` a mano puede dejar configuraciones inconsistentes.

6.3. Configuración de FreeRTOS: FreeRTOSConfig.h

En FreeRTOS estándar, toda la configuración del kernel se concentra en el archivo `FreeRTOSConfig.h`, que el desarrollador escribe y coloca en su proyecto (frecuencia del tick, cantidad de prioridades, funciones habilitadas, tamaño del heap, etc.). En ESP-IDF el archivo existe, pero con dos diferencias importantes:

- **No está en el proyecto**, sino dentro de la instalación de ESP-IDF, como parte del componente `freertos`.
- **No debe modificarse.** La documentación de Espressif lo declara privado: los valores configurables se toman de las opciones `CONFIG_FREERTOS_...` que se eligen en menuconfig.

Su ubicación, relativa a la carpeta de instalación de ESP-IDF (por ejemplo `C:\Espressif\frameworks\esp-idf-v6.x\`, según la versión y la ruta elegidas en el EIM), es:

```
esp-idf\components\freertos\
├── config\
│   ├── include\freertos\
│   │   └── FreeRTOSConfig.h            Configuración común del kernel
│   └── xtensa\include\freertos\
│       └── FreeRTOSConfig_arch.h       Parte específica de la arquitectura Xtensa (ESP32-S3)
├── FreeRTOS-Kernel\                    Código fuente del kernel (tasks.c, queue.c, timers.c...)
└── Kconfig                             Define las opciones de menuconfig de FreeRTOS
```

Una forma rápida de llegar al archivo desde VS Code es abrir cualquier fuente del proyecto, hacer `Ctrl+clic` sobre `#include "freertos/FreeRTOS.h"` y, dentro de ese archivo, sobre la inclusión de `FreeRTOSConfig.h`.

Dentro de `FreeRTOSConfig.h` se ve cómo cada parámetro clásico se vincula con menuconfig. Por ejemplo, `configTICK_RATE_HZ` se define como `CONFIG_FREERTOS_HZ`, y `configUSE_TIMERS` depende de `CONFIG_FREERTOS_USE_TIMERS`. Otros valores son fijos: `configMAX_PRIORITIES` vale 25 (prioridades de 0 a 24) y `configUSE_MUTEXES` vale 1.

Dónde se cambian los parámetros de FreeRTOS

En menuconfig, dentro de **Component config > FreeRTOS**, en dos subsecciones: **Kernel** (opciones del núcleo) y **Port** (opciones del puerto al hardware, como el temporizador del tick o el stack de interrupciones). Otras opciones relacionadas están en **Component config > ESP System Settings**. Las más relevantes para la materia son:

Opción (CONFIG_...)	Por defecto	Significado
`FREERTOS_HZ`	100	Frecuencia del tick en Hz (`configTICK_RATE_HZ`). 100 Hz = un tick cada 10 ms.
`FREERTOS_UNICORE`	No	Ejecutar FreeRTOS en un solo núcleo (el 0).
`FREERTOS_USE_TIMERS`	Sí	Habilita los software timers (`configUSE_TIMERS`).
`FREERTOS_TIMER_TASK_PRIORITY`	1	Prioridad de la tarea de servicio de timers (Tmr Svc).
`FREERTOS_TIMER_TASK_STACK_DEPTH`	2048	Stack de la tarea Tmr Svc, en bytes.
`FREERTOS_TIMER_QUEUE_LENGTH`	10	Largo de la cola de comandos de los timers.
`FREERTOS_CHECK_STACKOVERFLOW`	Canary	Método de detección de desborde de stack.
`FREERTOS_USE_TRACE_FACILITY`	No	Habilita funciones de diagnóstico, como `vTaskList()`.

Opción (CONFIG_...)	Por defecto	Significado
`ESP_MAIN_TASK_STACK_SIZE`	3584	Stack de la tarea que ejecuta `app_main()`, en bytes.
`ESP_MAIN_TASK_AFFINITY`	CPU0	Núcleo de la tarea que ejecuta `app_main()`.
`ESP_TASK_WDT_TIMEOUT_S`	5	Tiempo del Task Watchdog, en segundos.

Los valores por defecto pueden variar entre versiones de ESP-IDF; ante la duda, consultar la columna correspondiente en menuconfig.

> **Resumen:** en FreeRTOS estándar se edita `FreeRTOSConfig.h`; en ESP-IDF se usa **menuconfig**, que guarda los valores en `sdkconfig`; estos se convierten en macros en `build/config/sdkconfig.h`, y el `FreeRTOSConfig.h` de ESP-IDF las incluye para definir los parámetros `config...` del kernel.

7. El LED RGB direccionable (WS2812) y el periférico RMT

El LED de la placa no es un LED común que se enciende al poner un pin en alto. Es un **WS2812**: un encapsulado que contiene tres LEDs (rojo, verde y azul) y un pequeño **controlador** que recibe el color por un único cable de datos, mediante un protocolo serie.

- El color se envía como una trama de **24 bits**: 8 de verde, 8 de rojo y 8 de azul, en ese orden (formato **GRB**).
- Cada bit se codifica por el **ancho del pulso** en alto: un pulso corto (del orden de 0,4 µs) es un 0 y uno largo (del orden de 0,8 µs) es un 1, con un período de bit de aproximadamente 1,25 µs.
- Tras los 24 bits, una pausa en bajo de varias decenas de microsegundos (*reset* o *latch*) indica al controlador que aplique el nuevo color.
- El controlador **mantiene el último color recibido** y genera internamente el PWM de cada LED. Si nunca recibe una trama válida, puede quedar en un color arbitrario: es lo que ocurre con el pin mal configurado.
- Cada WS2812 retransmite por su salida los bits que no le corresponden, lo que permite **encadenar** muchos LEDs en un mismo cable. Así funcionan las tiras LED, y por eso el componente se llama `led_strip`.



Generar estos pulsos por software (*bit-banging*) sería frágil: cualquier interrupción o cambio de contexto del sistema operativo en medio de la trama la corrompe. Por eso se usa el periférico **RMT** (*Remote Control Transceiver*), que genera la forma de onda **por hardware**, a partir de una secuencia de pares (nivel, duración), sin intervención de la CPU. La **matriz de GPIO** del ESP32-S3 conecta la salida del RMT al pin 48. La alternativa **SPI** de menuconfig logra lo mismo codificando cada bit del WS2812 como un patrón de varios bits SPI.

En el código se usan tres funciones del componente: `led_strip_set_pixel(tira, indice, rojo, verde, azul)` escribe el color en un **buffer en memoria**; `led_strip_refresh(tira)` **transmite** ese buffer al LED por el RMT; y `led_strip_clear(tira)` apaga todos los LEDs.

> **Relación con la materia:** en un sistema con RTOS conviene delegar en periféricos las tareas con requisitos de temporización estrictos. La CPU queda libre para que el planificador ejecute otras tareas.

8. FreeRTOS en ESP-IDF

En ESP-IDF, FreeRTOS no es una biblioteca opcional: forma parte del framework y **se inicia automáticamente antes de que se ejecute cualquier código del usuario**. Por defecto, ESP-IDF usa su propia variante, llamada **IDF FreeRTOS**, basada en FreeRTOS v10.5.1 y modificada para funcionar en dos núcleos (sección 9).

8.1. Secuencia de arranque

Según la documentación de Espressif (*Application Startup Flow*), al energizar o reiniciar la placa ocurre lo siguiente:

1. **Bootloader de primera etapa (ROM):** código grabado en la ROM del chip, no modificable. Lee los pines de *strapping* para decidir el modo de arranque y carga desde la flash el bootloader de segunda etapa. Arranca solo el núcleo 0; el núcleo 1 permanece en reset.
2. **Bootloader de segunda etapa:** lee la tabla de particiones, carga la aplicación en RAM o la mapea desde la flash, verifica su integridad y salta a su punto de entrada.
3. **Inicialización de la aplicación:** el núcleo 0 configura CPU, relojes, memoria, heap, consola y componentes, y libera el reset del núcleo 1.
4. **Creación de la tarea principal e inicio del planificador** de FreeRTOS en ambos núcleos.
5. **Llamada a `app_main()`:** la tarea principal ejecuta la función `app_main()` del usuario.

8.2. app_main() se ejecuta dentro de una tarea

Aspecto	En ESP-IDF
Punto de entrada	No existe `main()`. El programador escribe `void app_main(void)`.
Planificador	Ya está corriendo. **Nunca** se llama a `vTaskStartScheduler()`.
Contexto	Se ejecuta dentro de una tarea FreeRTOS (en el log aparece como `main_task`), con prioridad, stack propio y estados como cualquier otra.
Prioridad	Fija: **1**, una por encima de la mínima.
Stack	`CONFIG_ESP_MAIN_TASK_STACK_SIZE`, en **bytes**.
Núcleo	`CONFIG_ESP_MAIN_TASK_AFFINITY`. Por defecto, **CPU0**. El log lo indica: *main_task: Started on CPU0*.
Retorno	**Permitido.** La tarea se elimina y el sistema sigue con las demás. Una tarea FreeRTOS común, en cambio, nunca debe retornar.

Existen entonces dos estilos válidos: usar `app_main()` **como una tarea más**, con su propio lazo infinito (ejemplo 1), o usarla **solo para inicializar**: configurar periféricos, crear las tareas de la aplicación y retornar (ejemplos 2 a 6).

8.3. Tareas que crea el sistema

Además de la tarea principal, ESP-IDF crea sus propias tareas. La documentación oficial enumera las siguientes:

Tarea	Núcleo	Prioridad	Función
IDLE0, IDLE1	Una por núcleo	0	Se ejecuta cuando no hay otra tarea lista en ese núcleo. Libera la memoria de tareas eliminadas.
main	CPU0 (config.)	1	Ejecuta `app_main()`. Se elimina al retornar.
Tmr Svc	CPU0	1 (config.)	Ejecuta las callbacks de los software timers. Se crea solo si se usa la API de timers.

Tarea	Núcleo	Prioridad	Función
ipc0, ipc1	Una por núcleo	24	Permiten que un núcleo pida ejecutar una función en el otro.
esp_timer	CPU0	22	Ejecuta las callbacks de los temporizadores de alta resolución de ESP-IDF.

Funciones como Wi-Fi o Bluetooth crean tareas adicionales. Notar que las tareas de usuario conviene que tengan prioridades bien por debajo de `ipc` y `esp_timer`.

8.4. El tick del sistema

Las funciones de tiempo de FreeRTOS reciben su argumento en **ticks**. La frecuencia del tick es `CONFIG_FREERTOS_HZ`, por defecto **100 Hz** (un tick cada 10 ms). La macro `pdMS_TO_TICKS(ms)` convierte milisegundos a ticks, y la constante `portTICK_PERIOD_MS` vale `1000 / CONFIG_FREERTOS_HZ`. La **resolución temporal** de `vTaskDelay()`, `vTaskDelayUntil()`, los timeouts y los software timers está limitada por el tick: con 100 Hz no es posible esperar 5 ms, porque el valor se redondea a un número entero de ticks.

9. FreeRTOS en dos núcleos (SMP)

El ESP32-S3 tiene **dos núcleos Xtensa LX7 idénticos**, que comparten la memoria y los periféricos. FreeRTOS estándar (*vanilla*) está pensado para **un solo núcleo**: en cada instante hay una única tarea en ejecución. IDF FreeRTOS lo extiende a **multiprocesamiento simétrico** (SMP, *Symmetric Multiprocessing*): un único kernel, con una única lista de tareas, planifica sobre los dos núcleos a la vez. En cada instante puede haber **dos tareas ejecutándose en paralelo**, una por núcleo.

9.1. Los núcleos

Núcleo	Nombre	Uso típico
0	PRO_CPU	Tareas de protocolo (Wi-Fi, Bluetooth), tareas del sistema (`esp_timer`, Tmr Svc) y, por defecto, `app_main()`.
1	APP_CPU	El resto de la aplicación.

Los nombres reflejan el uso habitual, pero no son una restricción: cualquier tarea puede correr en cualquier núcleo, salvo que se la fije a uno.

9.2. Afinidad: en qué núcleo corre una tarea

La **afinidad** de una tarea indica en qué núcleo puede ejecutarse. ESP-IDF agrega una función de creación con un parámetro extra:

```
BaseType_t xTaskCreatePinnedToCore(TaskFunction_t funcion, const char *nombre,
                                   uint32_t stack_bytes, void *parametro,
                                   UBaseType_t prioridad, TaskHandle_t *handle,
                                   BaseType_t nucleo);   // 0, 1 o tskNO_AFFINITY
```

- Con `nucleo` igual a **0** o **1**, la tarea queda **fijada** (*pinned*) a ese núcleo: solo se ejecuta allí.
- Con **`tskNO_AFFINITY`**, la tarea puede ejecutarse en **cualquiera** de los dos y cambiar de núcleo entre una ejecución y otra.
- La función estándar `xTaskCreate()` sigue disponible y equivale a usar `tskNO_AFFINITY`.
- `xPortGetCoreID()` devuelve el núcleo en el que se está ejecutando el código que la llama (lo usa el ejemplo 2).

9.3. Planificación en dos núcleos

- **Cada núcleo planifica de forma independiente**: elige la tarea lista de mayor prioridad que tenga permitido ejecutar (por su afinidad) y que no esté ya corriendo en el otro núcleo.
- En consecuencia, **una tarea de menor prioridad puede estar ejecutándose mientras otra de mayor prioridad espera**, si la de mayor prioridad está fijada al otro núcleo y este está ocupado. En un solo núcleo esto no ocurre.
- Cuando una tarea sin afinidad de mayor prioridad pasa a estado lista, se desaloja solo a uno de los núcleos, con preferencia por el actual.
- El reparto de tiempo entre tareas de igual prioridad (*time slicing*) es de tipo *best effort round robin*, no estrictamente equitativo. Para un round robin ideal, Espressif recomienda fijar al mismo núcleo todas las tareas de igual prioridad.
- Cada núcleo tiene su interrupción de tick, pero el **núcleo 0** se encarga de la cuenta del tiempo; el núcleo 1 solo la usa para el *time slicing*.
- Hay **una tarea IDLE por núcleo** (IDLE0 e IDLE1), fijadas a su núcleo.

9.4. Exclusión mutua en dos núcleos

En un solo núcleo, una forma de proteger una sección crítica muy corta es deshabilitar las interrupciones: así no puede haber cambio de contexto. **En dos núcleos eso no alcanza**, porque deshabilitar las interrupciones de un núcleo no impide que el otro, que sigue ejecutando otra tarea, acceda al mismo dato al mismo tiempo. Por eso IDF FreeRTOS modifica la API de secciones críticas para que reciba un ***spinlock***:

```
static portMUX_TYPE mi_spinlock = portMUX_INITIALIZER_UNLOCKED;

taskENTER_CRITICAL(&mi_spinlock);   // Deshabilita interrupciones en este núcleo
                                    // y toma el spinlock (excluye al otro núcleo)
/* ... sección crítica MUY corta, sin llamadas que bloqueen ... */
taskEXIT_CRITICAL(&mi_spinlock);
```

Un spinlock es un lock con **espera activa**: si el otro núcleo lo tiene, este núcleo queda girando en un lazo hasta que se libere. Por eso solo se usa para secciones de pocos microsegundos. Para proteger recursos durante más tiempo, o cuando se necesita bloquear la tarea, se usan **mutex** o **semáforos**, que funcionan correctamente en ambos núcleos (ejemplos 3 y 4).

Del mismo modo, `vTaskSuspendAll()` solo suspende el planificador del núcleo que la llama; la documentación advierte que **no es un mecanismo válido de exclusión mutua** en SMP.

9.5. Otras diferencias

- **Stack en bytes:** el tamaño de stack de `xTaskCreate()` se expresa en **bytes**, no en palabras.
- **Punto flotante:** una tarea que usa `float` queda fijada automáticamente al núcleo en el que estaba, porque el estado de la FPU no puede trasladarse de un núcleo al otro. El tipo `double` se emula por software y no tiene esta restricción.
- **Memoria:** las funciones de memoria de FreeRTOS (`pvPortMalloc()`, etc.) usan el heap de ESP-IDF (`heap_caps_malloc()`), no los esquemas heap_1 a heap_5 de FreeRTOS estándar.
- **Modo de un solo núcleo:** con `CONFIG_FREERTOS_UNICORE`, FreeRTOS corre solo en el núcleo 0 y se comporta como la versión estándar. Puede ser útil para comparar comportamientos.
- **Amazon SMP FreeRTOS:** ESP-IDF ofrece además, de forma experimental, la implementación SMP oficial de FreeRTOS (`CONFIG_FREERTOS_SMP`). En esta materia se usa la implementación por defecto, IDF FreeRTOS.

9.6. Resumen comparativo

Aspecto	FreeRTOS estándar	IDF FreeRTOS (ESP32-S3)
Núcleos	Uno	Dos (SMP)
Tareas en ejecución simultánea	Una	Hasta dos, una por núcleo
Inicio	`main()` crea tareas y llama a `vTaskStartScheduler()`	Planificador iniciado por ESP-IDF; el usuario escribe `app_main()`
Creación de tareas	`xTaskCreate()`	`xTaskCreate()` (sin afinidad) y `xTaskCreatePinnedToCore()`
Stack de tareas	En palabras	En bytes
Tareas IDLE	Una	Una por núcleo
Secciones críticas	`taskENTER_CRITICAL()` deshabilita interrupciones	`taskENTER_CRITICAL(&spinlock)`: interrupciones del núcleo + spinlock
Prioridad más alta lista	Siempre está en ejecución	No necesariamente: depende de su afinidad y del otro núcleo
Configuración	`FreeRTOSConfig.h` del proyecto	menuconfig (`FreeRTOSConfig.h` interno, no se edita)

9.7. En qué núcleo corre cada ejemplo

Ejemplo	Tareas de usuario	Núcleo
1	Tarea principal (`app_main()`)	**CPU0**, por `CONFIG_ESP_MAIN_TASK_AFFINITY`.
2	`tarea_led`, `tarea_mensajes`	**Cualquiera** (creadas con `xTaskCreate()`). El ejemplo imprime el núcleo en cada mensaje y puede variar.
3	`productora`, `display`	**Cualquiera.** Pueden correr en paralelo, una en cada núcleo.
4	`productora`, `display`, `calculo`	**CPU1**, las tres, con `xTaskCreatePinnedToCore()`. Es imprescindible para que se produzca la inversión de prioridad.
5	`productora`, `display`	**Cualquiera.**
6	Las del ejemplo 5 + callback del timer	Las tareas, en cualquiera. La callback corre dentro de **Tmr Svc**, en **CPU0**.

10. Ejemplo 1: blink con app_main() como tarea

10.1. Objetivo

Verificar el entorno y observar que, aunque el código no crea tareas ni inicia el planificador, **el programa ya corre sobre FreeRTOS**: `app_main()` se ejecuta dentro de la tarea principal y usa `vTaskDelay()` para bloquearse. Es el ejemplo oficial *blink* de ESP-IDF, sin modificaciones (solo la configuración de la sección 5.3).

10.2. Código

```
/* Blink Example

   This example code is in the Public Domain (or CC0 licensed, at your option.)

   Unless required by applicable law or agreed to in writing, this
   software is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
   CONDITIONS OF ANY KIND, either express or implied.
*/
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "led_strip.h"
#include "sdkconfig.h"

static const char *TAG = "example";

/* Use project configuration menu (idf.py menuconfig) to choose the GPIO to blink,
   or you can edit the following line and set a number here.
*/
#define BLINK_GPIO CONFIG_BLINK_GPIO

static uint8_t s_led_state = 0;

#ifdef CONFIG_BLINK_LED_STRIP

static led_strip_handle_t led_strip;

static void blink_led(void)
{
    /* If the addressable LED is enabled */
    if (s_led_state) {
        /* Set the LED pixel using RGB from 0 (0%) to 255 (100%) for each color */
        led_strip_set_pixel(led_strip, 0, 16, 16, 16);
        /* Refresh the strip to send data */
        led_strip_refresh(led_strip);
    } else {
        /* Set all LED off to clear all pixels */
        led_strip_clear(led_strip);
    }
}

static void configure_led(void)
{
    ESP_LOGI(TAG, "Example configured to blink addressable LED!");
    /* LED strip initialization with the GPIO and pixels number*/
    led_strip_config_t strip_config = {
        .strip_gpio_num = BLINK_GPIO,
        .max_leds = 1, // at least one LED on board
    };
#if CONFIG_BLINK_LED_STRIP_BACKEND_RMT
    led_strip_rmt_config_t rmt_config = {
        .resolution_hz = 10 * 1000 * 1000, // 10MHz
        .flags.with_dma = false,
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config, &rmt_config, &led_strip));
#elif CONFIG_BLINK_LED_STRIP_BACKEND_SPI
    led_strip_spi_config_t spi_config = {
```

```
        .spi_bus = SPI2_HOST,
        .flags.with_dma = true,
    };
    ESP_ERROR_CHECK(led_strip_new_spi_device(&strip_config, &spi_config, &led_strip));
#else
#error "unsupported LED strip backend"
#endif
    /* Set all LED off to clear all pixels */
    led_strip_clear(led_strip);
}

#elif CONFIG_BLINK_LED_GPIO

static void blink_led(void)
{
    /* Set the GPIO level according to the state (LOW or HIGH)*/
    gpio_set_level(BLINK_GPIO, s_led_state);
}

static void configure_led(void)
{
    ESP_LOGI(TAG, "Example configured to blink GPIO LED!");
    gpio_reset_pin(BLINK_GPIO);
    /* Set the GPIO as a push/pull output */
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
}

#else
#error "unsupported LED type"
#endif

void app_main(void)
{

    /* Configure the peripheral according to the LED type */
    configure_led();

    while (1) {
        ESP_LOGI(TAG, "Turning the LED %s!", s_led_state == true ? "ON" : "OFF");
        blink_led();
        /* Toggle the LED state */
        s_led_state = !s_led_state;
        vTaskDelay(CONFIG_BLINK_PERIOD / portTICK_PERIOD_MS);
    }
}
```

10.3. Explicación

- Las directivas `#ifdef CONFIG_BLINK_LED_STRIP` / `#elif CONFIG_BLINK_LED_GPIO` seleccionan, **en tiempo de compilación**, qué versión de `blink_led()` y `configure_led()` se compila, según lo elegido en menuconfig. Con nuestra configuración se compila la variante *LED strip* con RMT.
- `configure_led()` crea el *handle* del LED sobre el RMT y el GPIO configurado, y lo apaga. Se ejecuta una sola vez.
- `blink_led()` enciende el LED en blanco tenue (16 sobre 255 en cada color) o lo apaga, según `s_led_state`.
- `while (1)` es el lazo infinito típico del cuerpo de una tarea FreeRTOS: `app_main()` se comporta como una tarea más.
- `ESP_LOGI(TAG, ...)` imprime un mensaje de nivel *Info* con marca de tiempo y etiqueta.
- `vTaskDelay(CONFIG_BLINK_PERIOD / portTICK_PERIOD_MS)` es la línea clave. **Bloquea** la tarea durante 1000 / 10 = 100 ticks. Mientras está bloqueada no consume CPU y el planificador ejecuta otras tareas o, si no hay ninguna lista, la tarea IDLE del núcleo. Una forma equivalente y más usual es `vTaskDelay(pdMS_TO_TICKS(1000))`.

10.4. Salida esperada

```
I (281) main_task: Started on CPU0
I (291) main_task: Calling app_main()
I (291) example: Example configured to blink addressable LED!
I (301) example: Turning the LED OFF!
I (1301) example: Turning the LED ON!
I (2301) example: Turning the LED OFF!
```

Las dos primeras líneas las escribe ESP-IDF, no el ejemplo: muestran que existe la tarea principal, que corre en el **núcleo 0** y que es la que llama a `app_main()`.

10.5. Preguntas

1. ¿Qué ocurre si se agrega un `ESP_LOGI()` después del `while (1)`? ¿Y si se elimina el `while`?
2. Configurar `Blink period in ms` en 15 y medir el período real con las marcas de tiempo. Explicar la diferencia.
3. Reemplazar `vTaskDelay()` por un lazo de espera activa (un `for` vacío muy largo). ¿Qué informa el monitor al cabo de unos segundos? ¿Por qué?

11. Ejemplo 2: dos tareas

11.1. Objetivo

Crear tareas propias. Una tarea cambia el estado del LED cada 500 ms y otra envía un mensaje por consola cada 1000 ms. `app_main()` deja de ser una tarea de la aplicación: solo inicializa, crea las tareas y **retorna**.

11.2. Código

```
/*
 * Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
 * Ejemplo 2: blink con dos tareas FreeRTOS sobre ESP32-S3
 *
 * - tarea_led:      cambia el estado del LED RGB cada 500 ms.
 * - tarea_mensajes: envía un mensaje por consola cada 1000 ms.
 *
 * app_main() solo inicializa, crea las tareas y retorna.
 */

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "led_strip.h"
#include "sdkconfig.h"

#define PIN_LED              CONFIG_BLINK_GPIO   // Se configura en menuconfig (48 en la placa N16R8)
#define BRILLO               16                  // 0..255 por color

#define PERIODO_LED_MS       500
#define PERIODO_MENSAJE_MS   1000

#define STACK_TAREAS         4096                // En ESP-IDF el stack se expresa en BYTES
#define PRIORIDAD_LED        2
#define PRIORIDAD_MENSAJES   2

static const char *TAG = "dos_tareas";
static led_strip_handle_t tira_led;

static void configurar_led(void)
{
    led_strip_config_t config_tira = {
        .strip_gpio_num = PIN_LED,
        .max_leds = 1,                           // La placa tiene un solo LED
    };
    led_strip_rmt_config_t config_rmt = {
        .resolution_hz = 10 * 1000 * 1000,       // 10 MHz: 0,1 us por tick del RMT
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&config_tira, &config_rmt, &tira_led));
    led_strip_clear(tira_led);
}

static void tarea_led(void *parametros)
{
    int led_encendido = 0;

    while (1) {
        if (led_encendido == 0) {
            led_strip_set_pixel(tira_led, 0, BRILLO, BRILLO, BRILLO);
            led_strip_refresh(tira_led);
            led_encendido = 1;
        } else {
            led_strip_clear(tira_led);
            led_encendido = 0;
        }

        vTaskDelay(pdMS_TO_TICKS(PERIODO_LED_MS));
    }
}
```

```
static void tarea_mensajes(void *parametros)
{
    int contador = 0;

    while (1) {
        ESP_LOGI(TAG, "Mensaje %d - tick %lu - nucleo %d",
                 contador, (unsigned long) xTaskGetTickCount(), xPortGetCoreID());
        contador = contador + 1;

        vTaskDelay(pdMS_TO_TICKS(PERIODO_MENSAJE_MS));
    }
}

void app_main(void)
{
    BaseType_t resultado;

    configurar_led();

    resultado = xTaskCreate(tarea_led, "tarea_led", STACK_TAREAS, NULL, PRIORIDAD_LED, NULL);
    if (resultado != pdPASS) {
        ESP_LOGE(TAG, "No se pudo crear tarea_led");
    }

    resultado = xTaskCreate(tarea_mensajes, "tarea_mensajes", STACK_TAREAS, NULL, PRIORIDAD_MENSAJES,
NULL);
    if (resultado != pdPASS) {
        ESP_LOGE(TAG, "No se pudo crear tarea_mensajes");
    }

    ESP_LOGI(TAG, "Tareas creadas. app_main() termina y main_task se elimina.");
}
```

11.3. Explicación

Constantes

- `STACK_TAREAS 4096`: stack de cada tarea, en **bytes**. 4 KB alcanza holgadamente para `ESP_LOGI()`, que internamente usa un formateo tipo `printf`.
- `PRIORIDAD_LED` y `PRIORIDAD_MENSAJES` valen 2, una más que la tarea principal (1). Así, apenas se crea cada tarea, el planificador puede ejecutarla, antes incluso de que `app_main()` termine.

Las tareas

- La firma `void tarea(void *parametros)` es obligatoria para toda tarea: recibe un puntero genérico y no devuelve nada. El cuerpo es un lazo infinito; una tarea **nunca debe retornar**.
- `led_encendido` es una variable **local**: vive en el stack de la tarea. Cada tarea tiene su propio stack, por lo que estas variables no interfieren entre sí.
- `vTaskDelay(pdMS_TO_TICKS(PERIODO_LED_MS))` bloquea la tarea 500 ms (50 ticks).
- `xTaskGetTickCount()` devuelve los ticks transcurridos desde el inicio del planificador. Permite ver que entre mensajes pasan 100 ticks.
- `xPortGetCoreID()` devuelve el núcleo en el que se ejecuta la tarea en ese momento. Como las tareas se crean con `xTaskCreate()`, **sin afinidad**, pueden ejecutarse en cualquiera de los dos y el valor puede cambiar entre mensajes.
- El cast `(unsigned long)` hace coincidir el tipo `TickType_t` con el formato `%lu` y evita un warning.

app_main()

- `xTaskCreate(funcion, nombre, stack, parametro, prioridad, handle)` crea la tarea. El nombre solo se usa para depuración; el último argumento `NULL` indica que no se guarda el *handle* porque no se va a referenciar la tarea después.

- El retorno se compara con `pdPASS`: si no hay heap suficiente para el stack y el bloque de control (TCB), la creación falla.
- Al retornar `app_main()`, la tarea principal se elimina, pero las dos tareas creadas siguen ejecutándose.

11.4. Salida esperada

```
I (301) dos_tareas: Mensaje 0 - tick 30 - nucleo 1
I (301) dos_tareas: Tareas creadas. app_main() termina y main_task se elimina.
I (1301) dos_tareas: Mensaje 1 - tick 130 - nucleo 0
I (2301) dos_tareas: Mensaje 2 - tick 230 - nucleo 1
```

Los valores exactos varían. Lo importante es que **"Mensaje 0" aparece antes que el mensaje de `app_main()`**, porque la tarea nueva tiene más prioridad que la principal; que entre mensajes pasan siempre 100 ticks; y que el núcleo puede cambiar.

11.5. Preguntas

1. ¿Cambia algo si una tarea tiene más prioridad que la otra? ¿Por qué no se nota?
2. Crear las tareas con `xTaskCreatePinnedToCore(..., 1)`. ¿Qué núcleo informan ahora los mensajes?
3. ¿Qué pasaría si las dos tareas escribieran el LED, una cambiando el color? Anticipar el problema que se trabaja en los ejemplos siguientes.

12. Ejemplo 3: recurso compartido y semáforo binario

12.1. Objetivo y escenario

Mostrar una **condición de carrera** sobre un recurso compartido y cómo un **semáforo** la evita. El escenario es una estación de medición:

- **Recurso compartido:** una estructura global con una medición (número de muestra, temperatura y humedad).
- **Tarea productora** (sensor simulado): cada 300 ms genera la muestra N y escribe los tres campos **de a uno**, con una pausa de 20 ms entre cada uno que simula un acceso lento (por ejemplo, un sensor por I2C). Los valores se derivan de N: temperatura = 20 + N % 10, humedad = 50 + N % 10. No se conecta ningún hardware.
- **Tarea display** (consumidora): cada 100 ms lee la estructura y la muestra por consola. Como los valores dependen solo del número de muestra, el display puede **recalcular qué valores esperaba** y detectar una lectura inconsistente.

Con `USAR_SEMAFORO 0` el display lee a veces a mitad de una escritura; con `USAR_SEMAFORO 1` las lecturas son siempre coherentes.

12.2. Conceptos

- **Condición de carrera:** el resultado depende del orden, no controlado, en que se intercalan dos tareas que acceden a un mismo dato.
- **Sección crítica:** fragmento de código que accede al recurso compartido y que no debe ser ejecutado por más de una tarea a la vez.
- **Semáforo binario:** objeto del kernel con dos estados, disponible o tomado. `xSemaphoreTake()` lo toma si está disponible o **bloquea** la tarea hasta que lo esté; `xSemaphoreGive()` lo libera y despierta a una tarea que lo esperaba.

12.3. Código

```
/*
 * Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
 * Ejemplo 3: protección de un recurso compartido con un SEMÁFORO BINARIO
 *
 * - tarea_productora: simula un sensor. Cada 300 ms genera una muestra nueva
 *                     y la escribe, campo por campo, en una estructura global.
 * - tarea_display:    cada 100 ms lee la estructura, la muestra por consola
 *                     y verifica que los tres campos sean coherentes entre sí.
 *
 * USAR_SEMAFORO = 0 -> sin protección: aparecen datos inconsistentes.
 * USAR_SEMAFORO = 1 -> con semáforo binario: los datos siempre son coherentes.
 */

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_log.h"

#define USAR_SEMAFORO          0       // Cambiar a 1 y volver a grabar

#define PERIODO_PRODUCTOR_MS   300
#define PERIODO_DISPLAY_MS     100
#define DEMORA_ESCRITURA_MS    20      // Simula un acceso lento entre campo y campo

#define STACK_TAREAS           4096    // En bytes
#define PRIORIDAD_PRODUCTORA   2
#define PRIORIDAD_DISPLAY      2

static const char *TAG = "semaforo";
```

```
// ---------- Recurso compartido ----------
typedef struct {
    int numero_muestra;
    int temperatura;
    int humedad;
} medicion_t;

// Se inicializa con valores coherentes para la muestra 0
static medicion_t dato_compartido = { 0, 20, 50 };

static SemaphoreHandle_t semaforo;

// ---------- Valores simulados ----------
// Temperatura y humedad dependen solo del número de muestra.
// Así el display puede recalcular qué valores esperaba y detectar errores.
static int calcular_temperatura(int muestra)
{
    return 20 + (muestra % 10);        // 20..29
}

static int calcular_humedad(int muestra)
{
    return 50 + (muestra % 10);        // 50..59
}

// ---------- Acceso al recurso ----------
static void tomar_recurso(void)
{
#if USAR_SEMAFORO
    xSemaphoreTake(semaforo, portMAX_DELAY);   // Bloquea hasta obtenerlo
#endif
}

static void liberar_recurso(void)
{
#if USAR_SEMAFORO
    xSemaphoreGive(semaforo);
#endif
}

// ---------- Tareas ----------
static void tarea_productora(void *parametros)
{
    int muestra = 0;

    while (1) {
        muestra = muestra + 1;

        tomar_recurso();
        // ----- Inicio de la sección crítica -----
        dato_compartido.numero_muestra = muestra;
        vTaskDelay(pdMS_TO_TICKS(DEMORA_ESCRITURA_MS));
        dato_compartido.temperatura = calcular_temperatura(muestra);
        vTaskDelay(pdMS_TO_TICKS(DEMORA_ESCRITURA_MS));
        dato_compartido.humedad = calcular_humedad(muestra);
        // ----- Fin de la sección crítica -----
        liberar_recurso();

        vTaskDelay(pdMS_TO_TICKS(PERIODO_PRODUCTOR_MS));
    }
}

static void tarea_display(void *parametros)
{
    medicion_t copia;
    int lecturas = 0;
    int inconsistentes = 0;

    while (1) {
        // Se copia el dato y se libera el recurso enseguida.
        // La impresión, que es lenta, se hace fuera de la sección crítica.
```

```
        tomar_recurso();
        copia = dato_compartido;
        liberar_recurso();

        lecturas = lecturas + 1;

        if (copia.temperatura == calcular_temperatura(copia.numero_muestra) &&
            copia.humedad == calcular_humedad(copia.numero_muestra)) {
            ESP_LOGI(TAG, "Muestra %3d | T = %d C | H = %d %%",
                     copia.numero_muestra, copia.temperatura, copia.humedad);
        } else {
            inconsistentes = inconsistentes + 1;
            ESP_LOGW(TAG, "Muestra %3d | T = %d C | H = %d %%  <<< DATO INCONSISTENTE (%d de %d
lecturas)",
                     copia.numero_muestra, copia.temperatura, copia.humedad,
                     inconsistentes, lecturas);
        }

        vTaskDelay(pdMS_TO_TICKS(PERIODO_DISPLAY_MS));
    }
}

// ---------- Programa principal ----------
void app_main(void)
{
#if USAR_SEMAFORO
    semaforo = xSemaphoreCreateBinary();
    if (semaforo == NULL) {
        ESP_LOGE(TAG, "No se pudo crear el semaforo");
        return;
    }
    // Un semáforo binario se crea VACÍO (tomado).
    // Para usarlo como protección, hay que darlo una vez: el recurso queda libre.
    xSemaphoreGive(semaforo);
    ESP_LOGI(TAG, "Modo: CON semaforo binario");
#else
    ESP_LOGI(TAG, "Modo: SIN proteccion");
#endif

    xTaskCreate(tarea_productora, "productora", STACK_TAREAS, NULL, PRIORIDAD_PRODUCTORA, NULL);
    xTaskCreate(tarea_display, "display", STACK_TAREAS, NULL, PRIORIDAD_DISPLAY, NULL);
}
```

12.4. Explicación

- `#include "freertos/semphr.h"`: header de la API de semáforos y mutex.
- `dato_compartido` se inicializa con valores coherentes para la muestra 0; de lo contrario, la primera lectura daría inconsistente.
- `calcular_temperatura()` y `calcular_humedad()` las usan las dos tareas: la productora para generar los datos y el display para verificarlos.
- `tomar_recurso()` y `liberar_recurso()` encapsulan el semáforo. Con `USAR_SEMAFORO 0` quedan vacías, de modo que el resto del código es idéntico en ambos modos y la comparación es justa.
- `xSemaphoreTake(semaforo, portMAX_DELAY)`: `portMAX_DELAY` significa esperar indefinidamente. Con un valor en ticks, la función devolvería `pdFALSE` al vencer el plazo.
- En la productora, las tres escrituras y las dos pausas quedan entre tomar y liberar: es la **sección crítica**.
- En el display, `copia = dato_compartido;` copia la estructura completa **dentro** de la sección crítica; la verificación y la impresión, que son lentas, se hacen **fuera**, sobre la copia. El recurso debe quedar tomado el mínimo tiempo indispensable.
- `ESP_LOGW()` imprime con nivel *Warning*, en amarillo, para distinguir las inconsistencias.
- `xSemaphoreCreateBinary()` crea el semáforo **vacío** (tomado). Por eso se hace un `xSemaphoreGive()` inicial: sin esa línea, la primera tarea que intente tomarlo se bloquea para siempre.

12.5. Salida esperada

Con `USAR_SEMAFORO 0` aparecen, mezcladas con las lecturas correctas, líneas como:

```
W (...) semaforo: Muestra  17 | T = 26 C | H = 56 %  <<< DATO INCONSISTENTE (4 de 31 lecturas)
```

Es la muestra 17 con temperatura y humedad de la 16: el display leyó después de que se escribiera el número de muestra y antes de que se escribieran los otros campos. Como el productor tarda 40 ms en escribir y repite cada 340 ms, y el display lee cada 100 ms, aproximadamente una de cada ocho lecturas cae en la ventana de escritura. Con `USAR_SEMAFORO 1` no aparece ninguna inconsistencia.

> **Observación:** las pausas dentro de la sección crítica son artificiales: agrandan la ventana de la condición de carrera para que sea visible siempre. En un sistema real la ventana es de microsegundos, pero existe, y el error aparece de forma esporádica y difícil de reproducir. En el ESP32-S3, además, las dos tareas pueden correr **simultáneamente en núcleos distintos**, de modo que el problema puede ocurrir aun sin cambio de contexto.

12.6. Preguntas

1. Comentar el `xSemaphoreGive()` inicial. ¿Qué se observa? ¿Por qué?
2. ¿Por qué no se usa la consola como recurso compartido? Pista: `ESP_LOGI()` ya tiene un lock interno.
3. ¿Qué impide que el display, por error, libere el semáforo sin haberlo tomado? ¿Qué consecuencias tendría?

13. Ejemplo 4: inversión de prioridad y mutex

13.1. Objetivo

Mostrar la **inversión de prioridad**, un problema que el semáforo binario no resuelve, y cómo el **mutex** lo corrige mediante la **herencia de prioridad**.

13.2. Conceptos

- **Inversión de prioridad:** una tarea de prioridad alta espera un recurso que tiene una tarea de prioridad baja; mientras tanto, una tarea de prioridad **media**, que ni siquiera usa el recurso, desaloja a la baja y le impide terminar. En la práctica, la tarea alta queda esperando a la media, y la espera deja de estar acotada.
- **Mutex** (*mutual exclusion*): semáforo especializado en proteger recursos. Se diferencia del binario en dos aspectos. Tiene **dueño**: solo debe liberarlo la tarea que lo tomó. Tiene **herencia de prioridad**: cuando una tarea se bloquea esperando el mutex, la que lo posee sube temporalmente a la prioridad de la que espera, y vuelve a la suya al liberarlo.
- Regla práctica: **mutex para exclusión mutua, semáforo binario para sincronización** (avisar que ocurrió un evento, por ejemplo desde una interrupción).

13.3. Escenario

Para que la inversión ocurra en todos los ciclos, y no por azar, las tres tareas se despiertan cada 2 s con un desfase fijo, y **las tres se fijan al núcleo 1**:

Tiempo	Semáforo binario (USAR_MUTEX 0)	Mutex (USAR_MUTEX 1)
0 ms	BAJA toma el recurso y empieza a escribir (100 ms de CPU).	Igual.
20 ms	ALTA pide el recurso y se bloquea.	ALTA pide el recurso y se bloquea. **BAJA hereda la prioridad ALTA.**
40 ms	MEDIA se despierta y **desaloja a BAJA**: 500 ms de cálculo.	MEDIA se despierta, pero BAJA tiene ahora más prioridad: **no la desaloja**.
≈100 ms	BAJA sigue frenada.	BAJA termina, libera y vuelve a su prioridad. **ALTA lee enseguida.**
≈540 ms	MEDIA termina; BAJA retoma lo que le faltaba.	MEDIA ejecuta su cálculo.
≈600 ms	BAJA libera y **recién ahí ALTA lee**.	—
Espera de ALTA	**≈ 580 ms**	**≈ 80 ms**

Los tiempos son aproximados: dependen de la resolución del tick (10 ms) y del tiempo que insumen los mensajes de log.

13.4. Código

```
/*
 * Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
 * Ejemplo 4: inversión de prioridad. Semáforo binario vs. mutex
 *
 * Tres tareas, todas en el NÚCLEO 1, que se despiertan cada 2 segundos
 * con un pequeño desfase entre ellas:
 *
 *   tarea_productora (prioridad BAJA,  t =  0 ms): toma el recurso y tarda
 *                     100 ms de CPU en escribir una medición.
 *   tarea_display    (prioridad ALTA,  t = 20 ms): quiere leer la medición
 *                     y mide cuánto tiempo tuvo que esperar el recurso.
 *   tarea_calculo    (prioridad MEDIA, t = 40 ms): no usa el recurso;
 *                     hace 500 ms de cálculo intensivo.
```

```
 *
 * USAR_MUTEX = 0 -> semáforo binario: el display espera unos 580 ms
 *                   (inversión de prioridad).
 * USAR_MUTEX = 1 -> mutex con herencia de prioridad: el display espera
 *                   unos 80 ms.
 */

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_log.h"
#include "esp_timer.h"
#include "esp_rom_sys.h"

#define USAR_MUTEX              0       // Cambiar a 1 y volver a grabar

#define PERIODO_CICLO_MS        2000    // Las tres tareas repiten cada 2 s

#define DESFASE_DISPLAY_MS      20      // El display arranca 20 ms después de la productora
#define DESFASE_CALCULO_MS      40      // El cálculo arranca 40 ms después de la productora

#define TRABAJO_ESCRITURA_MS    50      // CPU entre campo y campo (100 ms en total)
#define TRABAJO_CALCULO_MS      500     // CPU que consume la tarea de prioridad media

#define PRIORIDAD_BAJA          2       // tarea_productora
#define PRIORIDAD_MEDIA         3       // tarea_calculo
#define PRIORIDAD_ALTA          4       // tarea_display

#define NUCLEO_TAREAS           1       // Las tres tareas comparten el núcleo 1
#define STACK_TAREAS            4096    // En bytes

static const char *TAG = "mutex";

// ---------- Recurso compartido ----------
typedef struct {
    int numero_muestra;
    int temperatura;
    int humedad;
} medicion_t;

static medicion_t dato_compartido = { 0, 20, 50 };

static SemaphoreHandle_t proteccion;     // Semáforo binario o mutex, según USAR_MUTEX

// ---------- Trabajo de CPU simulado ----------
// Ocupa la CPU durante 'milisegundos' de ejecución efectiva, en bloques de 1 ms.
// A diferencia de vTaskDelay(), la tarea NO se bloquea: sigue lista y consumiendo CPU.
// Si otra tarea la desaloja, el trabajo pendiente se retoma al volver.
static void trabajo_cpu(int milisegundos)
{
    int i;

    for (i = 0; i < milisegundos; i++) {
        esp_rom_delay_us(1000);          // Espera activa de 1 ms
    }
}

// ---------- Tareas ----------
static void tarea_productora(void *parametros)
{
    int muestra = 0;
    TickType_t ultimo_despertar = xTaskGetTickCount();

    while (1) {
        muestra = muestra + 1;

        xSemaphoreTake(proteccion, portMAX_DELAY);
        // ----- Inicio de la sección crítica -----
        dato_compartido.numero_muestra = muestra;
```

```
        trabajo_cpu(TRABAJO_ESCRITURA_MS);
        dato_compartido.temperatura = 20 + (muestra % 10);
        trabajo_cpu(TRABAJO_ESCRITURA_MS);
        dato_compartido.humedad = 50 + (muestra % 10);
        // ----- Fin de la sección crítica -----
        xSemaphoreGive(proteccion);

        ESP_LOGI(TAG, "[BAJA ] productora: libera el recurso (muestra %d)", muestra);

        vTaskDelayUntil(&ultimo_despertar, pdMS_TO_TICKS(PERIODO_CICLO_MS));
    }
}

static void tarea_display(void *parametros)
{
    medicion_t copia;
    int64_t tiempo_antes;
    int64_t tiempo_despues;
    int espera_ms;
    TickType_t ultimo_despertar;

    vTaskDelay(pdMS_TO_TICKS(DESFASE_DISPLAY_MS));
    ultimo_despertar = xTaskGetTickCount();

    while (1) {
        ESP_LOGI(TAG, "[ALTA ] display: pide el recurso");

        tiempo_antes = esp_timer_get_time();          // Tiempo en microsegundos
        xSemaphoreTake(proteccion, portMAX_DELAY);
        tiempo_despues = esp_timer_get_time();

        copia = dato_compartido;
        xSemaphoreGive(proteccion);

        espera_ms = (int) ((tiempo_despues - tiempo_antes) / 1000);

        ESP_LOGW(TAG, "[ALTA ] display: muestra %d | T = %d C | H = %d %% | ESPERO %d ms",
                 copia.numero_muestra, copia.temperatura, copia.humedad, espera_ms);

        vTaskDelayUntil(&ultimo_despertar, pdMS_TO_TICKS(PERIODO_CICLO_MS));
    }
}

static void tarea_calculo(void *parametros)
{
    TickType_t ultimo_despertar;

    vTaskDelay(pdMS_TO_TICKS(DESFASE_CALCULO_MS));
    ultimo_despertar = xTaskGetTickCount();

    while (1) {
        ESP_LOGI(TAG, "[MEDIA] calculo: inicio");
        trabajo_cpu(TRABAJO_CALCULO_MS);
        ESP_LOGI(TAG, "[MEDIA] calculo: fin");

        vTaskDelayUntil(&ultimo_despertar, pdMS_TO_TICKS(PERIODO_CICLO_MS));
    }
}

// ---------- Programa principal ----------
void app_main(void)
{
#if USAR_MUTEX
    proteccion = xSemaphoreCreateMutex();        // El mutex se crea DISPONIBLE
    ESP_LOGI(TAG, "Modo: MUTEX (con herencia de prioridad)");
#else
    proteccion = xSemaphoreCreateBinary();       // El semáforo binario se crea VACÍO...
    if (proteccion != NULL) {
        xSemaphoreGive(proteccion);              // ...por eso se lo da una vez
    }
    ESP_LOGI(TAG, "Modo: SEMAFORO BINARIO");
```

```
#endif

    if (proteccion == NULL) {
        ESP_LOGE(TAG, "No se pudo crear la proteccion del recurso");
        return;
    }

    // Las tres tareas se fijan al mismo núcleo. Si pudieran ejecutarse en núcleos
    // distintos, la tarea media no le quitaría la CPU a la baja y no habría inversión.
    xTaskCreatePinnedToCore(tarea_productora, "productora", STACK_TAREAS, NULL,
                            PRIORIDAD_BAJA, NULL, NUCLEO_TAREAS);
    xTaskCreatePinnedToCore(tarea_display, "display", STACK_TAREAS, NULL,
                            PRIORIDAD_ALTA, NULL, NUCLEO_TAREAS);
    xTaskCreatePinnedToCore(tarea_calculo, "calculo", STACK_TAREAS, NULL,
                            PRIORIDAD_MEDIA, NULL, NUCLEO_TAREAS);
}
```

13.5. Explicación

- `xSemaphoreCreateMutex()` crea un mutex, que **nace disponible**: no requiere el `xSemaphoreGive()` inicial. Se toma y se libera con las mismas funciones que el semáforo, por eso el resto del código no cambia entre los dos modos; lo que cambia es el comportamiento interno del kernel.
- `xTaskCreatePinnedToCore(..., NUCLEO_TAREAS)` fija las tres tareas al núcleo 1. **Es imprescindible:** con dos núcleos disponibles, la tarea media se ejecutaría en el otro y la baja no perdería la CPU. En multinúcleo, la inversión de prioridad se produce entre tareas que compiten por **el mismo núcleo**.
- `trabajo_cpu()` usa `esp_rom_delay_us(1000)`, una **espera activa** de 1 ms, repetida N veces. A diferencia de `vTaskDelay()`, la tarea **no se bloquea**: sigue lista y consumiendo CPU, como si calculara. Esto es esencial: la inversión ocurre porque la tarea baja *necesita CPU* para terminar y la media se la quita. Se trabaja en bloques de 1 ms para que, si la tarea es desalojada, el trabajo pendiente se retome al volver.
- `vTaskDelayUntil(&ultimo_despertar, periodo)` espera hasta un instante **absoluto** (`ultimo_despertar + periodo`), no un intervalo desde ahora como `vTaskDelay()`. El período resulta exacto aunque el cuerpo de la tarea tarde distinto en cada ciclo; sin esto, los desfases se correrían y el escenario no se repetiría. La variable se inicializa con `xTaskGetTickCount()` y la función la actualiza en cada llamada.
- El `vTaskDelay()` al comienzo del display y del cálculo produce los desfases de 20 y 40 ms.
- `esp_timer_get_time()` devuelve el tiempo desde el arranque en **microsegundos**; se usa para medir la espera del display con más resolución que el tick.

13.6. Salida esperada (semáforo binario)

```
I (2300) mutex: [ALTA ] display: pide el recurso
I (2320) mutex: [MEDIA] calculo: inicio
I (2820) mutex: [MEDIA] calculo: fin
W (2880) mutex: [ALTA ] display: muestra 2 | T = 22 C | H = 52 % | ESPERO 580 ms
I (2880) mutex: [BAJA ] productora: libera el recurso (muestra 2)
```

Con mutex, la línea del display aparece **antes** de "calculo: inicio" e informa una espera de aproximadamente 80 ms.

13.7. Preguntas

1. Llevar `TRABAJO_CALCULO_MS` a 1000. ¿Cómo cambia la espera del display en cada modo? ¿Quién controla esa espera con el semáforo binario?
2. Reemplazar `xTaskCreatePinnedToCore(..., 1)` por `xTaskCreate()`. ¿Se sigue produciendo la inversión? Explicar.
3. Con mutex, ¿qué ocurre si la tarea display llama a `xSemaphoreGive()` sin haber tomado el mutex?

14. Ejemplo 5: cola de mensajes

14.1. Objetivo

Resolver el mismo problema productor/consumidor con una **cola de mensajes**, el mecanismo de comunicación entre tareas más usado en FreeRTOS. La productora genera una medición cada 500 ms y la envía a la cola; el display se bloquea esperando mensajes y muestra cada uno al llegar.

14.2. Conceptos

- Una **cola** es un buffer FIFO de capacidad fija, administrado por el kernel, de elementos de tamaño fijo.
- Los datos se pasan **por copia**: al enviar, el dato se copia dentro de la cola; al recibir, se copia en una variable del receptor. **No hay variable global compartida** y, por lo tanto, no existe el dato a medio escribir.
- La cola resuelve a la vez **la transferencia del dato**, la **exclusión mutua** (internamente está protegida) y la **sincronización**: el receptor se bloquea mientras la cola está vacía y despierta cuando llega un mensaje, sin *polling*. Del mismo modo, un emisor puede bloquearse mientras la cola está llena.

14.3. Código

```
/*
 * Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
 * Ejemplo 5: productor y consumidor con una COLA DE MENSAJES
 *
 * - tarea_productora: cada 500 ms genera una medición y la ENVÍA a la cola.
 * - tarea_display:    se bloquea esperando mensajes en la cola y, cuando
 *                     llega uno, lo muestra por consola.
 *
 * No hay variables globales compartidas: la cola COPIA cada mensaje.
 *
 * Para ver qué pasa cuando el consumidor es más lento que el productor,
 * cambiar DEMORA_DISPLAY_MS a 800 y volver a grabar.
 */

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"

#define LARGO_COLA              5       // Cantidad máxima de mensajes en la cola

#define PERIODO_PRODUCTOR_MS    500
#define DEMORA_DISPLAY_MS       0       // Probar con 800: display más lento que el productor
#define ESPERA_ENVIO_MS         0       // 0 = si la cola está llena, se descarta la muestra

#define STACK_TAREAS            4096    // En bytes
#define PRIORIDAD_PRODUCTORA    2
#define PRIORIDAD_DISPLAY       2

static const char *TAG = "cola";

// ---------- Mensaje ----------
typedef struct {
    int numero_muestra;
    int temperatura;
    int humedad;
    TickType_t tick_produccion;         // Momento en que se generó el dato
} medicion_t;

static QueueHandle_t cola_mediciones;

// ---------- Tareas ----------
```

```
static void tarea_productora(void *parametros)
{
    medicion_t medicion;
    int muestra = 0;
    int perdidas = 0;
    BaseType_t resultado;

    while (1) {
        muestra = muestra + 1;

        // El mensaje se arma en una variable LOCAL de la productora
        medicion.numero_muestra = muestra;
        medicion.temperatura = 20 + (muestra % 10);
        medicion.humedad = 50 + (muestra % 10);
        medicion.tick_produccion = xTaskGetTickCount();

        // xQueueSend COPIA la estructura dentro de la cola
        resultado = xQueueSend(cola_mediciones, &medicion, pdMS_TO_TICKS(ESPERA_ENVIO_MS));

        if (resultado == pdPASS) {
            ESP_LOGI(TAG, "Productora: envia muestra %d (en cola: %d de %d)",
                     muestra, (int) uxQueueMessagesWaiting(cola_mediciones), LARGO_COLA);
        } else {
            perdidas = perdidas + 1;
            ESP_LOGE(TAG, "Productora: COLA LLENA, se pierde la muestra %d (perdidas: %d)",
                     muestra, perdidas);
        }

        vTaskDelay(pdMS_TO_TICKS(PERIODO_PRODUCTOR_MS));
    }
}

static void tarea_display(void *parametros)
{
    medicion_t recibida;
    int antiguedad_ms;

    while (1) {
        // Se bloquea hasta que haya un mensaje. No consume CPU mientras espera.
        xQueueReceive(cola_mediciones, &recibida, portMAX_DELAY);

        antiguedad_ms = (int) ((xTaskGetTickCount() - recibida.tick_produccion) * portTICK_PERIOD_MS);

        ESP_LOGW(TAG, "DISPLAY -> Muestra %3d | T = %d C | H = %d %% | antiguedad %d ms",
                 recibida.numero_muestra, recibida.temperatura, recibida.humedad, antiguedad_ms);

        // Simula un display lento (por ejemplo, un LCD que tarda en refrescar)
        if (DEMORA_DISPLAY_MS > 0) {
            vTaskDelay(pdMS_TO_TICKS(DEMORA_DISPLAY_MS));
        }
    }
}

// ---------- Programa principal ----------
void app_main(void)
{
    // Cola de LARGO_COLA elementos, cada uno del tamaño de una medicion_t
    cola_mediciones = xQueueCreate(LARGO_COLA, sizeof(medicion_t));
    if (cola_mediciones == NULL) {
        ESP_LOGE(TAG, "No se pudo crear la cola");
        return;
    }

    xTaskCreate(tarea_productora, "productora", STACK_TAREAS, NULL, PRIORIDAD_PRODUCTORA, NULL);
    xTaskCreate(tarea_display, "display", STACK_TAREAS, NULL, PRIORIDAD_DISPLAY, NULL);
}
```

14.4. Explicación

- `#include "freertos/queue.h"`: header de la API de colas.
- `xQueueCreate(LARGO_COLA, sizeof(medicion_t))` crea una cola de 5 elementos del tamaño de la estructura. La memoria para todos se reserva de una vez. Devuelve `NULL` si no hay memoria.
- `tick_produccion` lleva en el mensaje el instante en que se generó el dato; el display lo usa para calcular su **antigüedad**, es decir, cuánto tiempo esperó en la cola.
- `xQueueSend(cola, &medicion, espera)` copia `sizeof(medicion_t)` bytes desde `&medicion` al final de la cola. Después de llamarla, la productora puede modificar `medicion` sin afectar lo enviado. Si la cola está llena, espera el tiempo indicado: con 0 no espera y devuelve `errQUEUE_FULL`, de modo que la muestra se pierde (se informa con `ESP_LOGE()`, en rojo). Con `portMAX_DELAY`, la productora se bloquearía hasta que haya lugar.
- `uxQueueMessagesWaiting(cola)` devuelve cuántos mensajes hay en la cola.
- `xQueueReceive(cola, &recibida, portMAX_DELAY)` extrae el mensaje más antiguo y lo copia en `recibida`. Si la cola está vacía, **la tarea se bloquea** sin consumir CPU hasta que llegue uno.
- `DEMORA_DISPLAY_MS` simula un display lento, por ejemplo un LCD que tarda en refrescar.

14.5. Salida esperada

Con `DEMORA_DISPLAY_MS 0`, el display es más rápido que la productora y la cola nunca acumula:

```
I (810) cola: Productora: envia muestra 2 (en cola: 0 de 5)
W (810) cola: DISPLAY -> Muestra   2 | T = 22 C | H = 52 % | antiguedad 0 ms
```

Que se informe "en cola: 0" justo después de enviar se explica porque el display ya retiró el mensaje antes de que la productora consultara el nivel.

Con `DEMORA_DISPLAY_MS 800` entran dos mensajes por segundo y salen 1,25: el nivel de la cola sube hasta 5, la antigüedad de los datos crece hasta unos 4 s, y con la cola llena aparecen en rojo los mensajes `COLA LLENA, se pierde la muestra N`; en el display se ven saltos en la numeración.

14.6. Para discutir

- **Una cola absorbe diferencias transitorias de velocidad, no permanentes.** Si el consumidor es siempre más lento, la cola se llena, cualquiera sea su largo; agrandarla solo demora el problema y aumenta la latencia.
- **¿Perder datos o frenar al productor?** Es una decisión de diseño: con espera infinita no se pierde nada, pero la productora queda atada al ritmo del display, lo que puede ser inaceptable para un sensor que debe muestrear a tiempo fijo.
- **Último valor:** si al display solo le interesa el dato más reciente, puede usarse una cola de largo 1 con `xQueueOverwrite()` (un "buzón").
- **Copia por valor o por puntero:** aquí se copian 16 bytes. Para datos grandes conviene encolar punteros, pero entonces vuelve el problema de quién es dueño de la memoria.

15. Ejemplo 6: cola de mensajes y software timer

15.1. Objetivo

Partiendo del ejemplo 5 sin cambios, agregar un **software timer** de FreeRTOS que cambie el estado del LED cada 1000 ms, **sin crear una tarea** para ello.

15.2. Conceptos

- Un **software timer** ejecuta una función (*callback*) cuando vence un período. Puede ser **periódico** (*auto-reload*), que se recarga solo, o de **un solo disparo** (*one-shot*).
- La callback **no es una tarea**: la ejecuta la **tarea de servicio de timers** de FreeRTOS (**Tmr Svc**), que en ESP-IDF corre en el **núcleo 0** con prioridad 1 por defecto. Todos los timers del sistema comparten esa tarea.
- Las funciones de la API (`xTimerStart()`, `xTimerStop()`, etc.) no actúan directamente: **envían un comando a Tmr Svc a través de una cola interna**, la *timer command queue*. Es decir, los timers están construidos sobre el mismo mecanismo de colas del ejemplo 5.
- La resolución de los timers es la del **tick** (10 ms).

15.3. Código

```
/*
 * Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
 * Ejemplo 6: cola de mensajes + SOFTWARE TIMER
 *
 * - tarea_productora: cada 500 ms genera una medición y la ENVÍA a la cola.
 * - tarea_display:    se bloquea esperando mensajes en la cola y, cuando
 *                     llega uno, lo muestra por consola.
 * - timer_led:        software timer de FreeRTOS que cada 1000 ms ejecuta
 *                     una función que cambia el estado del LED RGB.
 *
 * No hay variables globales compartidas: la cola COPIA cada mensaje.
 *
 * Para ver qué pasa cuando el consumidor es más lento que el productor,
 * cambiar DEMORA_DISPLAY_MS a 800 y volver a grabar.
 */

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/timers.h"
#include "esp_log.h"
#include "led_strip.h"
#include "sdkconfig.h"

#define PIN_LED                 CONFIG_BLINK_GPIO   // Se configura en menuconfig (48 en la placa N16R8)
#define BRILLO                  16                  // 0..255 por color
#define PERIODO_LED_MS          1000

#define LARGO_COLA             5       // Cantidad máxima de mensajes en la cola

#define PERIODO_PRODUCTOR_MS    500
#define DEMORA_DISPLAY_MS       0       // Probar con 800: display más lento que el productor
#define ESPERA_ENVIO_MS         0       // 0 = si la cola está llena, se descarta la muestra

#define STACK_TAREAS            4096    // En bytes
#define PRIORIDAD_PRODUCTORA    2
#define PRIORIDAD_DISPLAY       2

static const char *TAG = "cola";

// ---------- Mensaje ----------
typedef struct {
    int numero_muestra;
```

```
    int temperatura;
    int humedad;
    TickType_t tick_produccion;         // Momento en que se generó el dato
} medicion_t;

static QueueHandle_t cola_mediciones;

// ---------- LED y timer ----------
static led_strip_handle_t tira_led;
static TimerHandle_t timer_led;
static int led_encendido = 0;            // Solo la usa la función del timer

static void configurar_led(void)
{
    led_strip_config_t config_tira = {
        .strip_gpio_num = PIN_LED,
        .max_leds = 1,                          // La placa tiene un solo LED
    };
    led_strip_rmt_config_t config_rmt = {
        .resolution_hz = 10 * 1000 * 1000,      // 10 MHz: 0,1 us por tick del RMT
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&config_tira, &config_rmt, &tira_led));
    led_strip_clear(tira_led);
}

// Función de callback del timer.
// NO es una tarea: la ejecuta la tarea de servicio de timers de FreeRTOS
// ("Tmr Svc") cada vez que vence el período. Debe ser corta y no bloquearse.
static void callback_timer_led(TimerHandle_t timer)
{
    if (led_encendido == 0) {
        led_strip_set_pixel(tira_led, 0, BRILLO, BRILLO, BRILLO);
        led_strip_refresh(tira_led);
        led_encendido = 1;
    } else {
        led_strip_clear(tira_led);
        led_encendido = 0;
    }
}

// ---------- Tareas ----------
static void tarea_productora(void *parametros)
{
    medicion_t medicion;
    int muestra = 0;
    int perdidas = 0;
    BaseType_t resultado;

    while (1) {
        muestra = muestra + 1;

        // El mensaje se arma en una variable LOCAL de la productora
        medicion.numero_muestra = muestra;
        medicion.temperatura = 20 + (muestra % 10);
        medicion.humedad = 50 + (muestra % 10);
        medicion.tick_produccion = xTaskGetTickCount();

        // xQueueSend COPIA la estructura dentro de la cola
        resultado = xQueueSend(cola_mediciones, &medicion, pdMS_TO_TICKS(ESPERA_ENVIO_MS));

        if (resultado == pdPASS) {
            ESP_LOGI(TAG, "Productora: envia muestra %d (en cola: %d de %d)",
                     muestra, (int) uxQueueMessagesWaiting(cola_mediciones), LARGO_COLA);
        } else {
            perdidas = perdidas + 1;
            ESP_LOGE(TAG, "Productora: COLA LLENA, se pierde la muestra %d (perdidas: %d)",
                     muestra, perdidas);
        }

        vTaskDelay(pdMS_TO_TICKS(PERIODO_PRODUCTOR_MS));
    }
}
```

```
static void tarea_display(void *parametros)
{
    medicion_t recibida;
    int antiguedad_ms;

    while (1) {
        // Se bloquea hasta que haya un mensaje. No consume CPU mientras espera.
        xQueueReceive(cola_mediciones, &recibida, portMAX_DELAY);

        antiguedad_ms = (int) ((xTaskGetTickCount() - recibida.tick_produccion) * portTICK_PERIOD_MS);

        ESP_LOGW(TAG, "DISPLAY -> Muestra %3d | T = %d C | H = %d %% | antiguedad %d ms",
                 recibida.numero_muestra, recibida.temperatura, recibida.humedad, antiguedad_ms);

        // Simula un display lento (por ejemplo, un LCD que tarda en refrescar)
        if (DEMORA_DISPLAY_MS > 0) {
            vTaskDelay(pdMS_TO_TICKS(DEMORA_DISPLAY_MS));
        }
    }
}

// ---------- Programa principal ----------
void app_main(void)
{
    // Cola de LARGO_COLA elementos, cada uno del tamaño de una medicion_t
    cola_mediciones = xQueueCreate(LARGO_COLA, sizeof(medicion_t));
    if (cola_mediciones == NULL) {
        ESP_LOGE(TAG, "No se pudo crear la cola");
        return;
    }

    xTaskCreate(tarea_productora, "productora", STACK_TAREAS, NULL, PRIORIDAD_PRODUCTORA, NULL);
    xTaskCreate(tarea_display, "display", STACK_TAREAS, NULL, PRIORIDAD_DISPLAY, NULL);

    // LED y timer
    configurar_led();

    timer_led = xTimerCreate("timer_led",                    // Nombre (para depuración)
                             pdMS_TO_TICKS(PERIODO_LED_MS),  // Período
                             pdTRUE,                         // pdTRUE = periódico (auto-reload)
                             NULL,                           // ID del timer (no se usa)
                             callback_timer_led);            // Función a ejecutar
    if (timer_led == NULL) {
        ESP_LOGE(TAG, "No se pudo crear el timer");
        return;
    }

    // Crear el timer no lo arranca: hay que iniciarlo
    if (xTimerStart(timer_led, 0) != pdPASS) {
        ESP_LOGE(TAG, "No se pudo iniciar el timer");
    }
}
```

15.4. Explicación

- `#include "freertos/timers.h"`: header de la API de software timers. Se agregan además `led_strip.h`, `sdkconfig.h` y la función `configurar_led()` del ejemplo 2.
- `callback_timer_led(TimerHandle_t timer)`: su firma es obligatoria; recibe el *handle* del timer que venció, útil si una misma función atiende varios timers. **No tiene `while (1)` ni `vTaskDelay()`**: se ejecuta de principio a fin y retorna; la periodicidad la da el timer.
- `led_encendido` es una variable **global estática** y no local, porque la callback empieza de cero en cada llamada y el estado debe conservarse entre llamadas. No requiere protección porque solo la usa la callback.

- `xTimerCreate(nombre, periodo, auto_recarga, ID, callback)`: `pdTRUE` indica timer periódico; `pdFALSE` sería de un solo disparo. El ID es un puntero libre para asociar un dato al timer; aquí no se usa. Devuelve `NULL` si no hay memoria.
- `xTimerStart(timer_led, 0)`: **crear el timer no lo arranca**. El segundo argumento es cuánto esperar si la cola de comandos de Tmr Svc está llena.

Reglas para escribir una callback de timer

- **No debe bloquearse** (`vTaskDelay()`, esperas indefinidas en colas o semáforos): mientras se ejecuta, Tmr Svc está ocupada y **ningún otro timer del sistema se atiende**.
- Debe ser **breve**. `led_strip_refresh()` espera a que el RMT termine de transmitir, lo que demora decenas de microsegundos: es aceptable. Un trabajo largo debe derivarse a una tarea, por ejemplo enviando un mensaje por una cola.
- Cuidar el **stack**: Tmr Svc tiene 2048 bytes por defecto. Por eso la callback del ejemplo no usa `ESP_LOGI()`, cuyo formateo consume bastante stack.

15.5. Resultado esperado

El monitor muestra la misma salida que el ejemplo 5 y, en paralelo, el LED permanece 1 s encendido y 1 s apagado, sin que haya ninguna tarea de usuario dedicada a él.

15.6. Para discutir

- **Tarea vs. timer:** en el ejemplo 2, el LED requería una tarea con un stack de 4096 bytes; aquí alcanza con una función y la pequeña estructura del timer. Para acciones cortas y periódicas, el timer es más económico.
- **Prioridad de Tmr Svc:** con prioridad 1, por debajo de la productora y del display, una tarea de mayor prioridad que acapare el núcleo 0 retrasaría el LED.
- **Otras funciones:** `xTimerStop()`, `xTimerReset()` y `xTimerChangePeriod()`; por ejemplo, para que el LED parpadee más rápido cuando la cola se llena.
- **Alternativa de ESP-IDF:** `esp_timer`, con resolución de microsegundos y callbacks ejecutadas en la tarea `esp_timer` (prioridad 22). Conviene cuando se necesita más precisión que el tick, pero no es portable a otros microcontroladores con FreeRTOS.

16. Resumen de la API utilizada

Función / macro	Header	Uso
`xTaskCreate()`	task.h	Crea una tarea sin afinidad de núcleo.
`xTaskCreatePinnedToCore()`	task.h	Crea una tarea fijada a un núcleo (específica de ESP-IDF).
`vTaskDelay(ticks)`	task.h	Bloquea la tarea un intervalo relativo.
`vTaskDelayUntil(&ref, periodo)`	task.h	Bloquea hasta un instante absoluto: períodos exactos.
`xTaskGetTickCount()`	task.h	Ticks desde el inicio del planificador.
`pdMS_TO_TICKS(ms)`	FreeRTOS.h	Convierte milisegundos a ticks.
`xPortGetCoreID()`	FreeRTOS.h	Núcleo en el que se ejecuta el código.
`xSemaphoreCreateBinary()`	semphr.h	Crea un semáforo binario (vacío).
`xSemaphoreCreateMutex()`	semphr.h	Crea un mutex (disponible, con herencia de prioridad).

Función / macro	Header	Uso
`xSemaphoreTake(s, espera)`	semphr.h	Toma un semáforo o mutex; bloquea si no está disponible.
`xSemaphoreGive(s)`	semphr.h	Libera un semáforo o mutex.
`xQueueCreate(largo, tamano)`	queue.h	Crea una cola.
`xQueueSend(q, &dato, espera)`	queue.h	Copia un dato al final de la cola.
`xQueueReceive(q, &dato, espera)`	queue.h	Extrae el dato más antiguo; bloquea si la cola está vacía.
`uxQueueMessagesWaiting(q)`	queue.h	Cantidad de mensajes en la cola.
`xTimerCreate()`	timers.h	Crea un software timer.
`xTimerStart(t, espera)`	timers.h	Inicia un timer (envía un comando a Tmr Svc).
`ESP_LOGI/W/E(tag, ...)`	esp_log.h	Mensajes de log por consola, niveles Info, Warning y Error.
`esp_timer_get_time()`	esp_timer.h	Tiempo desde el arranque en microsegundos.
`esp_rom_delay_us(us)`	esp_rom_sys.h	Espera activa (no bloquea la tarea).

Las constantes más usadas son `portMAX_DELAY` (espera indefinida), `pdPASS` y `pdFAIL`, `pdTRUE` y `pdFALSE`, y `tskNO_AFFINITY`.

17. Problemas frecuentes

Síntoma	Causa probable y solución
EIM not found desde VS Code	El EIM no está instalado. Instalarlo por separado (sección 3.3).
Select current ESP-IDF version sin versiones; el New Project Wizard sin plantillas	Falta ESP-IDF o VS Code no se reinició tras instalarlo. Completar la sección 3.3 y reiniciar.
Errores de compilación con rutas extrañas	Proyecto en una ruta con espacios o acentos. Moverlo a `C:\esp\`.
La PC no detecta la placa	Cable de solo carga, o falta el driver del conversor USB-serie. Probar otro cable o el conector USB nativo.
Access denied sobre el puerto COM	Otro programa tiene el puerto abierto (Arduino IDE, PuTTY, otro monitor). Cerrarlo.
No logra grabar (*Failed to connect*)	Mantener **BOOT**, presionar y soltar **RST**, soltar **BOOT** y reintentar.
Hay mensajes en el monitor pero el LED queda fijo o apagado	Pin incorrecto: configurar **GPIO 48** en menuconfig (sección 5.3).
Rojo y verde intercambiados	LED con otro orden de color; se ajusta en la configuración de `led_strip`.
Mensaje del *Task Watchdog*	Una tarea ejecuta un lazo sin bloquearse nunca e impide que corra la tarea IDLE de su núcleo.
Stack overflow / reinicios con *backtrace*	Stack insuficiente en `xTaskCreate()` (recordar que es en bytes) o uso de `ESP_LOG` en una callback de timer.
Una tarea no hace nada al tomar un semáforo	Semáforo binario creado y nunca liberado inicialmente (falta el `xSemaphoreGive()`).

18. Actividades propuestas

1. **(Ej. 1)** Cambiar el período desde menuconfig, sin tocar el código, y verificarlo con las marcas de tiempo del monitor.
2. **(Ej. 1)** Configurar un período de 15 ms, medir el período real y explicar la diferencia en función del tick. Repetir con `CONFIG_FREERTOS_HZ` en 1000.
3. **(Ej. 2)** Modificar la tarea del LED para que recorra rojo, verde y azul. Fijar ambas tareas al núcleo 1 y verificar el núcleo informado.
4. **(Ej. 2)** Agregar una tercera tarea con espera activa de 6 s sin bloquearse y analizar el mensaje del Task Watchdog según el núcleo en que corra.
5. **(Ej. 3)** Ejecutar en ambos modos y registrar el porcentaje de lecturas inconsistentes. Calcular el valor teórico a partir de los tiempos del ejemplo.
6. **(Ej. 4)** Medir la espera del display con semáforo y con mutex para tres valores de `TRABAJO_CALCULO_MS` y graficar los resultados.
7. **(Ej. 4)** Repetir el experimento sin fijar las tareas a un núcleo y explicar el resultado.
8. **(Ej. 5)** Probar `ESPERA_ENVIO_MS` en 0 y en `portMAX_DELAY` con el display lento. Comparar la antigüedad de los datos y la cantidad de muestras perdidas.
9. **(Ej. 5)** Reemplazar la cola por un buzón de largo 1 con `xQueueOverwrite()` y comparar.
10. **(Ej. 6)** Usar `xTimerChangePeriod()` para que el LED parpadee cada 200 ms cuando la cola tenga 3 o más mensajes.
11. **(Ej. 6)** Hacer que el LED emita un destello corto (100 ms) cada segundo, combinando un timer periódico y uno de un solo disparo.

19. Referencias

- Espressif Systems. *ESP-IDF Programming Guide — Installation of ESP-IDF and Tools on Windows*. docs.espressif.com/projects/esp-idf/en/stable/get-started/windows-setup.html
- Espressif Systems. *ESP-IDF Installation Manager (EIM)*. docs.espressif.com/projects/idf-im-ui/en/latest/
- Espressif Systems. *Application Startup Flow (ESP32-S3)*. docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/startup.html
- Espressif Systems. *FreeRTOS Overview (ESP32-S3)*. docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos.html
- Espressif Systems. *FreeRTOS (IDF) — SMP changes*. docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos_idf.html
- Espressif Systems. *Project Configuration (Kconfig)*. docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/kconfig-reference.html
- Espressif Systems. Ejemplo *get-started/blink* y componente *led_strip*. github.com/espressif/esp-idf — components.espressif.com
- Barry, R. *Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide*. freertos.org
- FreeRTOS. *FreeRTOS Reference Manual* y documentación de la API. freertos.org
- Worldsemi. *WS2812B Intelligent control LED integrated light source — Datasheet*.